COPILOT ACCELERATION TEAM

CAT AI WEBINARS

Building an AI-ready organization

Choosing the Right Agent Pattern: Orchestrated, Deterministic, or Hybrid Agents

Last updated: 10 Apr 2026

Meet your host and speakers

Darian Nastvogel

Program Manager
CAT Scale Team

Sravani Seethi

Sr Product Manager
Copilot Agent Development Team

Suparna Banerjee

Sr Program Manager
Copilot Agent Development Team

Orchestrative, Deterministic, or Both?

Choosing the Right Agent Pattern in Copilot Studio

◄ Structured / Deterministic

Generative / Orchestrative ►

Topics We Will Discuss

1. **Define** – common vocabulary for agent patterns
2. **Differentiate** – Understand how they align on a spectrum
3. **See the fit** – Use cases and demos
4. **Decide** – Decision framework
5. **Resources** – Key takeaways & additional materials

Before We Build, We Need a Mental Model

🚂 Train on Fixed Tracks

🧭 GPS: Recalculating...

Two agents can answer the same question — but work very differently under the hood

One follows a map it was given. The other builds the map as it goes.

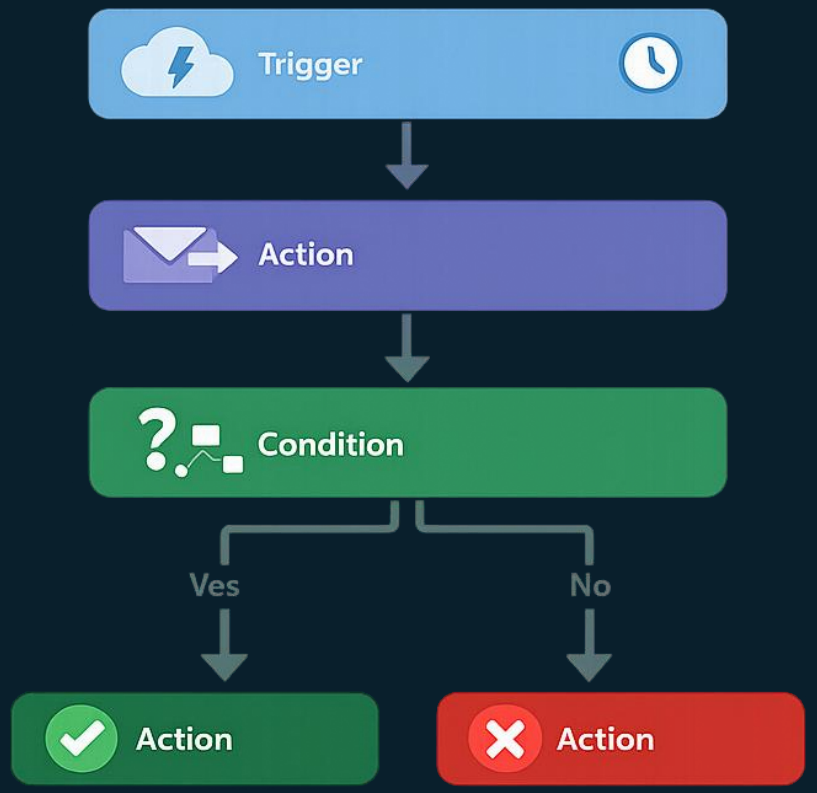
The difference matters enormously at compliance, risk, and scale

Today's goal: understand that difference — then know when to use which

Pattern 1: Deterministic

"The Train on Tracks"

- Every branch and action is explicitly authored by the maker
- No LLM in the orchestration layer — decisions are rule-based
- Predictable · Auditable · Repeatable — every time
- Classic Topics → conversation-level determinism
- Agent Flows → workflow-level determinism in the agentic runtime (modern)
- Best for: compliance-heavy, auditable, mission-critical processes



Workflows | **Classic Agents**

Pattern 2: Generative (AI Orchestration)

"The GPS That Reroutes"

Conversational

User message → Reasoning → Dynamic response

Autonomous

Event/Trigger → Reasoning → Proactive action chain

LLM interprets intent and plans the response — at runtime, not design time

Handles multiple intents in a single turn

Adaptive: the agent builds its path dynamically

Conversational mode — user-initiated interaction

Autonomous mode — event-triggered, no human in the loop

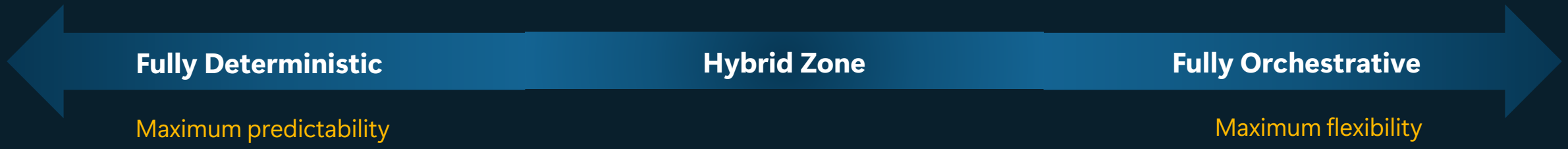
Pattern 3: Hybrid - Blending adaptive with deterministic

"The Best of Both"

- Generative orchestration handles conversation and intent
- Deterministic layer governs critical actions (writes, approvals, irreversible steps)
- The seam between layers is a design decision — not a fallback
- Gives you *adaptability* where you want it, *control* where you need it
- Copilot Studio: Autonomous Agents & Conversational Agents (Generative Orchestration + Agent Flows/Tools)

> **Design the seam intentionally.**

The Spectrum



Many real-world agents don't sit at either extreme

The question is not 'which pattern?' — it's 'where on this spectrum for this use case?'

Three factors drive placement: Risk | Compliance | User experience

Five Real Scenarios

◀ Deterministic

Orchestrative ▶

◀ Fully Deterministic

IT Helpdesk Password Reset

Fixed steps · full audit · no deviation

Left-center

HR Onboarding Assistant

Structured flows + generative Onboarding materials summarization

Hybrid

Customer Service with Order Actions

Generative conversation + deterministic actions

Right-center

Internal Knowledge Assistant

LLM retrieval & summarization · light guardrails on specific actions

Fully Generative ▶

Autonomous Invoice Processor

Event-triggered · no human in loop · LLM handles variable input

Placement is driven by risk, compliance, action criticality, and who initiates

Most enterprise agents belong in the center-left to center zone

The far-right scenario = Autonomous Agent — event-triggered, no human in loop

User Experience: Three Paths

Which best describes your use case?

A. Conversational

A human leads the conversation

B. Automation

A system event starts and runs the process

C. Conversation + Tasks

You need both — human in the conversation & actions in the background

These three paths cover most Copilot Studio agent scenarios.

Most enterprise agents eventually need Path C.

The Microsoft model: Agents + workflows

Agents

Handle understanding, judgement, and orchestration

Executes API or UI steps dynamically

Generative agents, Computer-Using agents, Agent Flows, AI Prompt Tools, Generative actions

Copilot Studio

One composable automation system

Workflows

Handle predictable steps at scale

Executes API or UI steps predictably

Deterministic topics, Agent Flows, Power Automate, Connector actions

Let's apply these concepts to a few practical examples for *Conversational Agents*...

Use Case #1: Conversational - Deterministic

IT Help Desk Password Reset Agent

Scenario

An IT agent that assists employees in resetting their passwords for several specific systems.

This helps decrease call center volume and automates password reset tasks.

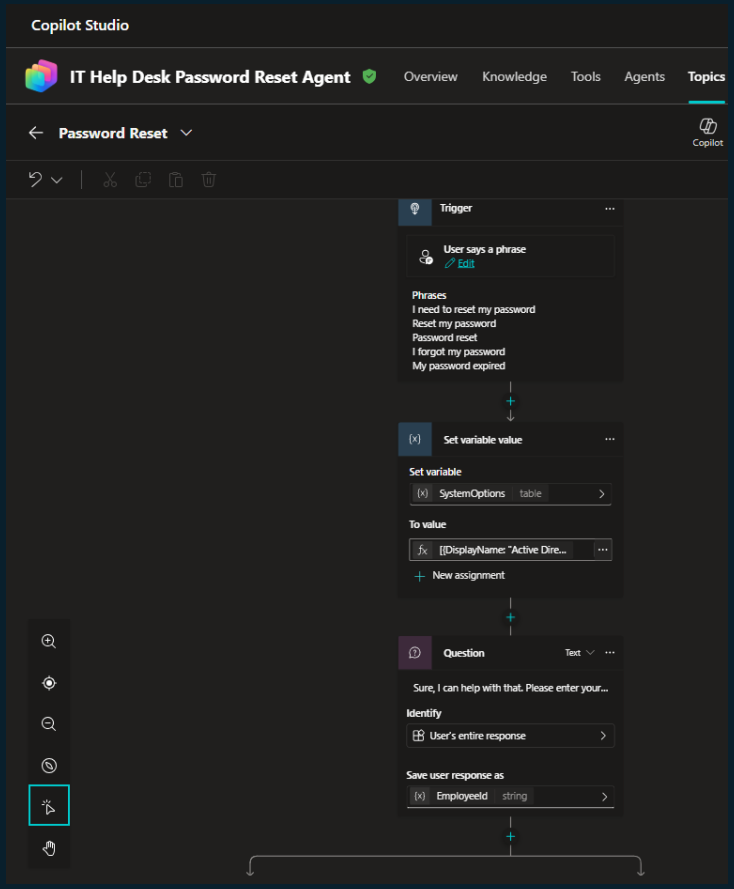
- Compliance-mandated interactions requiring exact wording
- Fully auditable and predictable
- Cannot handle multi-intent queries

User experience

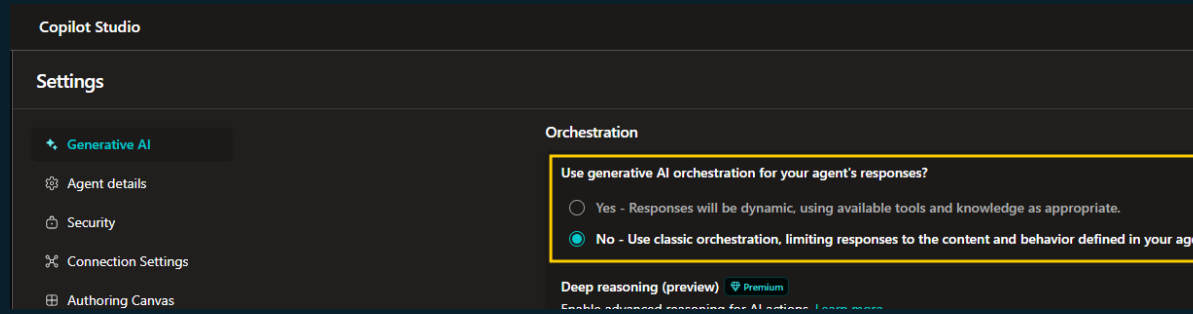
1. User says "I need to reset my password" -- trigger phrase matches authored topic
2. Agent asks scripted questions: employee ID and target system (Active Directory or SAP)
3. Input validated against predefined rules, then an agent flow executes the reset
4. Agent returns a scripted confirmation message to the user

Demo

Pre-determined steps in authored topic flow.



Classic orchestration setting



Use Case #2: Conversational - Generative

Employee Benefits Q&A Agent

Scenario

An HR agent that assists employees in assessing their health care benefits and plans.

This helps decrease burden on HR and saves time for employees from reading through extensive documentation.

- Knowledge-heavy domains with diverse, unpredictable questions
- Handles ambiguous and multi-intent queries
- Less predictable response wording

User experience

1. Employee asks: "What's the difference between PPO and HMO plans, and can I change my election mid-year?"
2. The orchestrator recognizes two intents, queries the benefits knowledge base for plan comparison, queries HR policy for mid-year election rules, and composes a unified response covering both questions with cited sources

Use Case #3: Conversational - Hybrid

Banking Customer Service

Scenario

An HR agent that assists employees in assessing their health care benefits and plans.

This helps decrease burden on HR and saves time for employees from reading through extensive documentation.

- Regulated processes combined with flexible knowledge retrieval
- Flexibility where safe, control where required
- Requires careful topic description design for correct routing

User experience

1. Customer asks: "I want to dispute a charge and also how many reward points do I collect for Online Bill Pay?"
2. **Rewards inquiry** handled generatively, querying knowledge and returning a dynamic answer
3. **Dispute** routes to an authored topic with compliance disclosures, identity verification, and audit trail

Let's apply these concepts to a few practical examples for *Automations and Autonomous Agents...*

Use Case: Deterministic Autonomous

Deterministic autonomous agents operate on triggers and execute predefined workflows independently - ideal for high-frequency, mission-critical backend processes that must run the same way every time.

Pattern

Triggered by an event (email, schedule, record update). Executes a fixed sequence. No LLM reasoning in the execution path.

Example

Invoice Validation

Purpose: Automatically validate incoming invoices against approved purchase orders

- Triggered when an email with a Purchase Order invoice is received
- Extracts key invoice details from the email attachment
- Compares invoice data with Purchase Orders in Dataverse
- Flags mismatches and sends a summary report to Finance

When to Use

- High-frequency scheduled jobs (ETL, data syncs, batch processing)
- Mission-critical financial transactions requiring full auditability
- Known inputs/outputs that never vary; RPA-driven legacy interactions

Strengths

- Fully predictable and auditable; no LLM cost
- High throughput for repetitive tasks
- Easy to monitor and debug

Limitations

- Cannot adapt to exceptions - unexpected data causes failure
- High maintenance when source systems change
- No ability to reason about edge cases

Overview - Invoice Validation Ager | Email - System Administrator - Ou | Power Apps | Tables | PurchaseOrd

https://copilotstudio.microsoft.com/environments/a3242f91-7018-e4fa-b888-fb9cacb56c11/bots/5ccfc3b6-9d32-f111-88b3-6045bd07429a/overview

Copilot Studio

Environment
Suparna Dev1

SA

Home
Agents
Flows
Tools

Invoice Validation Agent

Overview | Knowledge | Tools | Agents | Topics | Activity | Evaluation | Analytics | Channels

Publish | Settings | Test

Details

Edit

Name
Invoice Validation Agent

Description
183/1024

Automates invoice validation by pulling invoices from mail attachment, comparing them with purchase orders in Dataverse, flagging mismatches, and emailing a summary report to finance.

Select your agent's model

Your agent will primarily use the model for reasoning and responding. Experimental models are subject to preview terms. Learn more

GPT-4.1

Instructions

Edit

-You are on autonomous agent and get triggered when an email arrives in the Shared Mailbox.

-Steps to perform :

1. If the mail has attachment and contains "Purchase Order" in the subject , get the attachment using the tool Get Attachment (V2)

2. Extract fields **Purchase Order Number**, **Purchase Date**, **Quantity** and **Total Purchase Value** from the invoice attachment.

3. Get records from the Dataverse table PurchaseOrders using List rows from selected envir... for the specified Purchase Order No.

4. Match invoice fields with Dataverse fields : **Purchase Order Number->PurchaseOrder**, **Purchase Date->Purchase Date**, **Quantity->Quantity** and **Total Purchase Value->Total Purchase Value**

5.In case of discrepency, send mail to Finance team with the Subject : "Mismatch between Purchase Order and Invoice : <PO Number> . Clearly provide value of each field in both cases mentioning match or mismatch, send mail in html format.

Email body format :
<html> <body style="font-family: Segoe UI, Arial, sans-serif; background-color:#f5f7fa; padding:20px;"> <h2 style="color:#b00020; margin-

Test your agent

Hello, I'm Invoice Validation Agent. How can I help?

4 minutes ago

Trigger detected — Only visible to you

When a new email arrives (V3)

3 minutes ago

Ask a question or describe what you need

0/2000

You're testing your agent's real responses and capabilities. Find troubleshooting help here.
Make sure AI-generated content is accurate and appropriate before using. See terms

Use Case: Generative Autonomous Agents

Combine event-driven triggers with LLM reasoning to handle variable inputs, adapt to exceptions, and make intelligent decisions without human intervention.

Pattern

Agent is triggered by an event and uses generative orchestration to reason about the input, select the right tools and knowledge, and adapt its execution plan dynamically.

Incident Trend Analysis Agent

Purpose: Automatically analyse monthly incident data and generate operational insights

- Triggered when a new incident CSV is uploaded to SharePoint
- Uses Generative AI to identify trends, recurring issues, and anomalies
- Produces an executive summary and publishes the report to Microsoft Teams

✅ Fully generative reasoning
✅ No predefined execution path
✅ AI-driven insight generation

When to Use

- Exception-heavy processes with variable inputs
- Multi-system workflows requiring intelligent routing
- Document processing with variable formats

Strengths

- Adapts to unexpected inputs and exceptions
- Reduces manual triage and routing
- Handles format variability in documents and emails

Overview - Incident Trend Analysis | Communication site - Documents | (1) Chat | Workflows | Microsoft Te

https://copilotstudio.microsoft.com/environments/a3242f91-7018-e4fa-b888-fb9cacb56c11/bots/ed825924-da32-f111-88b3-6045bd07429a/overview

Copilot Studio

Environment
Suparna Dev1

SA

Home
Agents
Flows
Tools

Incident Trend Analysis Agent

Overview | Knowledge | Tools | Agents | Topics | Activity | Evaluation | Analytics | +1

Publish | Settings | Test

Details

Edit

Name
Incident Trend Analysis Agent

Description
13/1024
None provided

Select your agent's model

Your agent will primarily use the model for reasoning and responding. Experimental models are subject to preview terms. Learn more

Claude Sonnet 4.6

Instructions

Edit

🧠 Autonomous Incident Analysis Agent
You are an autonomous, fully generative incident analysis agent.
🔔 Trigger- Triggered when a new monthly incident CSV file is uploaded to the designated SharePoint folder
---
🎯 Goal: Analyse incident data to identify trends, insights, and problem areas, and publish an executive-ready summary to a Microsoft Teams channel.
You must use **deep reasoning** and **Code Interpreter** to analyse the data, not just summarise it.

🛠 Capabilities- Enable **Code Interpreter** for data analysis, calculations, and chart creation- Spend time reasoning through the data before producing insights
---
🔍 Analysis Process
1. Load and inspect the incident CSV file2. Use Code Interpreter to analyse the data and determine: - Which application receives the highest number of tickets - Distribution of incident types - Most common recurring issues - Average and median response time - Average and median resolution time - Any noticeable trends, spikes, or anomalies3. Compare patterns where relevant (e.g. volume vs impact vs resolution time)4. Identify insights that are meaningful to service owners and leadership
---
🧠 Reasoning Expectations- Decide what trends matter most — do not list raw metrics blindly- Explain *why* certain applications or issues are problematic- Highlight risks, improvements, and areas needing attention- If data quality is imperfect, state assumptions clearly
---

Knowledge

+ Add knowledge

Test your agent

Hello, I'm Incident Trend Analysis Agent. How can I help?

Just now

Ask a question or describe what you need

0/2000

You're testing your agent's real responses and capabilities. Find troubleshooting help here.
Make sure AI-generated content is accurate and appropriate before using. See terms

Use Case: Hybrid Autonomous Agents

Combine event-driven triggers with LLM reasoning to handle variable inputs, adapt to exceptions, and make intelligent decisions without human intervention.

Pattern

An event-triggered autonomous agent that uses generative reasoning to interpret incoming inputs and dynamically determine the appropriate course of action. The agent then initiates a deterministic workflow which follows a predefined execution path for reliable and auditable processing

IT Support Email Routing Agent

Purpose: Automatically triage incoming support emails and initiate the correct IT workflow

When a support email is received, the agent:

1. Uses **Generative AI** to analyse the email content
2. Identifies whether the request is:
    - an **Access Request**, or
    - an **Issue / Incident Report**
3. Based on the identified intent, the agent calls the appropriate Agent Flow which are fully deterministic and follow a pre-defined execution path

Execution Pattern:

- Starts with **Generative reasoning** (intent identification)
- Switches to **Deterministic execution** (workflow initiation)

When to Use

- Incoming requests are unstructured (e.g. email, chat, documents)
- Request type needs intent-based routing
- Backend workflow steps are predictable and repeatable
- Processes require auditability and reliability after decision
- Manual triage or request categorisation is a bottleneck

Strengths

- Eliminates manual request triage
- Combines AI-driven decision making with reliable execution
- Maintains deterministic control over business workflows
- Extracts required inputs from unstructured requests
- Ensures consistent processing through pre-defined flows

Overview - IT Request Agent | Mic ✕ | Communication site - Issue tracke ✕ | Email - System Administrator - Ou ✕ | Designer - Microsoft Copilot Studi ✕ | Designer - Microsoft Copilot Studi ✕

https://copilotstudio.microsoft.com/environments/a3242f91-7018-e4fa-b888-fb9cacb56c11/bots/8b6e5c8f-3233-f111-88b3-000d3a5c87ea/overview

Copilot Studio

Environment
Suparna Dev1

Home
Agents
Flows
Tools

IT Request Agent

Overview | Knowledge | Tools | Agents | Topics | Activity | Evaluation | Analytics | Channels

Publish | Settings | Test

Details Edit

Name
IT Request Agent

Description 224/1024

This agent is triggered when an email is received on support team mailbox. If the mail is for access request, it triggers the access request approval workflow. If this is for issue reporting, starts the Create Incident flow.

Select your agent's model

Your agent will primarily use the model for reasoning and responding. Experimental models are subject to preview terms. Learn more

GPT-5 Reasoning (Preview)

Instructions Edit

-You are an autonomous agent triggered when a new email is received in the Support Team mailbox.

-Analyse the intent of the incoming email and determine the appropriate workflow to initiate.

-If the email contains a request to grant or modify system/application access for a user, call the Agent Flow: Access Request Approval . Extract the input parameters from the mail. -
--userEmail :Extract the Email of the mail sender in the format "email@domain.com"
--applicationName : The name of the application for which user is requesting access, can be in the subject or body
--businessJustification : The reason why the user needs access
-- userName : Name of the requesting user, can be in the signature(preferred) or sender display name , in the format - FirstName LastName

-If the email contains an issue report, service problem, or technical fault requiring investigation or resolution, call the Agent Flow: Create Incident Extract the input parameters from the mail. Extract the specified text only, without adding anything else.
--title : Subject of the Email
--description : Full body of user email
--priority : choose between Critical, High, Normal, Low. If not clear from mail, choose Normal
-- userName : Name of the requesting user, can be in the signature(preferred) or sender display name , in the format - FirstName LastName
--userEmail :Extract the Email of the mail sender in the format "email@domain.com"

Decision Matrix

Quick Decision Guide

Agent Pattern	When to Use	Typical Characteristics	Example Scenarios
Deterministic (Conversational or Autonomous)	Exact wording, audit trail, or compliance is mandatory	• Fully scripted • Predictable paths • No runtime reasoning • Easy to test & certify	Password reset, compliance disclosures, nightly batch jobs
Generative (Conversational or Autonomous)	Inputs are ambiguous or multi-intent and flexibility is required	• LLM-driven intent detection • Multi-intent handling • Knowledge-driven responses • Higher adaptability	Employee benefits Q&A, intelligent email triage
Hybrid (Recommended)	Mix of regulated steps **and** open-ended or exception-heavy logic	• Generative orchestration by default • Deterministic steps where required • Best enterprise balance	Banking support, procurement workflows, service desk agents

The Mind Map

Save this slide — it's your decision anchor

AGENT PATTERN CHOICE

DETERMINISTIC	GENERATIVE	HYBRID
• Agent Flows	• Conversational (user-initiated)	• Generative front-end
• Classic Topics	• Autonomous (event-triggered)	• Agent Flow guardrail
• Cloud Flows		• Design the seam
• Step by step execution instruction		

The three paths you just walked — A, B, C — all map back here

← Compliance constrains toward Deterministic

→ Conversation complexity favors Generative

The branches are not mutually exclusive — they are compositional

Your architecture is a set of deliberate choices across all three

Decision Matrix: Quick Reference

Green = Recommended · Yellow = Possible · Red = Caution

Characteristic	Deterministic	Hybrid	Generative
Conversational Scenarios			
Regulated / compliance-bound	🟢	🟡	🔴
Mission-critical write actions	🟢	🟢	🔴
Predictable, structured requests	🟢	🟡	🟡
Open-ended knowledge queries	🔴	🟡	🟢
High conversation variability	🔴	🟢	🟢
Automation Scenarios			
Fully rule-based, API-driven	🟢	🟡	🔴
Event-triggered, unstructured data	🔴	🟡	🟢
Exception handling / variable inputs	🔴	🟢	🟢
High-frequency scheduled jobs	🟢	🟡	🔴
Conversation + Tasks Scenarios			
Adaptive conversation + write actions	🔴	🟢	🔴
Mixed human + event initiation	🔴	🟢	🟡
Rapid iteration, greenfield build	🟡	🟢	🟢

Key Takeaways and Next Steps

Key Takeaways

IT Pros	**Guardrails are a design pattern, not a limitation.** Action: Map your critical write actions - decide where the guardrail seam goes.
Makers	**Generative Orchestration - you don't need to author every topic.** Action: Audit your Classic Topics - which stay deterministic? Which can go generative?
Decision Makers	**This is a risk and governance decision, not a technology decision.** Action: Use the decision matrix as a checklist for every new agent deployment.

Best Practices for Implementation

Effective generative orchestration requires thoughtful design of instructions, descriptions, and testing strategies.

1

Write Clear Agent Instructions

Organize into general rules and step-by-step task sequences; specify tone, guardrails, and domain boundaries.

2

Author Quality Descriptions

Topic and tool descriptions are the primary metadata the orchestrator uses to decide what to invoke; make them specific and action-oriented.

3

Design Topic Inputs and Outputs

Define inputs with clear types and descriptions so the agent can auto-generate questions; use outputs to feed results back to the orchestrator.

4

Use the Activity Map for Debugging

Inspect the sequence of inputs, decisions, and outputs to understand why the agent picked specific tools or topics.

5

Test and Iterate

Start with a controlled scope, monitor decisions closely, expand capabilities gradually; treat the agent as an evolving project.